

1. การสร้าง Account

The screenshot shows the Postman interface for a POST request to `http://localhost:8080/accounts`. The request body is a JSON object with the following data:

```
1 {
2   "accountNumber": "6733802894",
3   "ownerName": "รสริน เมืองหงษ์",
4   "balance": 0
5 }
```

The response status is **200 OK** with a response time of 879 ms and a body size of 266 B. The response body is displayed in a table format:

id	1
accountNumber	6733802894
ownerName	รสริน เมืองหงษ์
balance	0

- อธิบายผล

1. ทดสอบการสร้างบัญชี Account โดยใช้คำสั่ง POST/accounts ผ่านโปรแกรม Postman
2. ส่งข้อมูล accountNumber, ownerName และ balance ไปยังระบบ
3. สามารถรับข้อมูลและสร้าง Account ใหม่สำเร็จ
4. ระบบกำหนด id ให้กับ Account อัตโนมัติจากฐานข้อมูล
5. ผลลัพธ์ที่ได้แสดงข้อมูลของ Account ที่สร้างขึ้น แสดงว่าระบบสามารถบันทึกข้อมูล Account ลงในฐานข้อมูล PostgreSQL ได้ถูกต้อง

2. การฝากเงินสำเร็จ

The screenshot shows the Postman interface for a POST request to `http://localhost:8080/accounts/1/deposit`. The request body is a JSON object: `{ "amount": 2500 }`. The response is a 200 OK status with a response time of 1.81 s and a body size of 197 B. The response body is a JSON object: `{ "message": "Desposit successful" }`. The interface includes tabs for Docs, Params, Authorization, Headers, Body, Scripts, and Settings. The Body tab is selected, and the response is displayed in the bottom panel.

```
POST http://localhost:8080/accounts/1/deposit
```

```
{  
  "amount": 2500  
}
```

200 OK • 1.81 s • 197 B • Save Response

```
{  
  "message": "Desposit successful"  
}
```

- อธิบายผล

- 1.ทดสอบการฝากเงินโดยใช้คำสั่ง `POST/accounts/{id}/depposit` ผ่าน Postman
- 2.ส่งจำนวนเงินที่ต้องการฝากในรูปแบบ JSON ตามตัวอย่างในภาพ
- 3.ระบบค้นหา Account จาก `accountId` ที่ระบุ
- 4.ระบบนำจำนวนเงินที่ฝากไปเพิ่มในค่า `balance` ของ Account
- 5.ระบบสร้างข้อมูล `DepositTransaction` เพื่อเก็บรายละเอียดของรายการฝากเงิน
- 6.ระบบตอบกลับข้อความ “Desposit successful” แสดงว่าการฝากเงินดำเนินการสำเร็จ
- 7.การทดลองแสดงให้เห็นว่าการอัปเดตยอดเงินและการสร้างรายการฝากเงินสามารถทำงานได้ตามที่ออกแบบไว้

3. ตรวจสอบ Account

HTTP Lab9 > accounts_byId Save Share

GET http://localhost:8080/accounts/1 Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body 200 OK • 386 ms • 276 B • Save Response

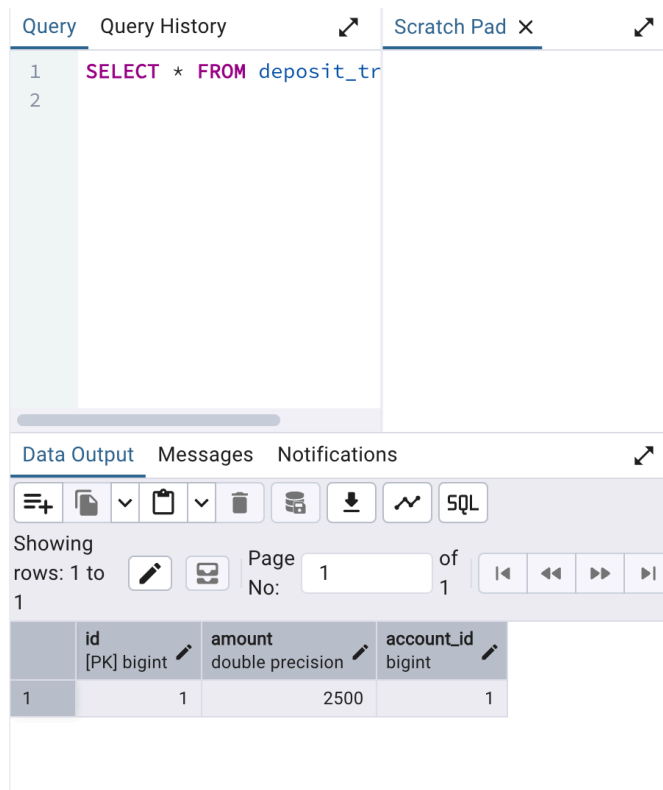
{ } JSON Preview Visualize

id	1
accountNumber	6733802894
ownerName	รสริน เมืองหงษ์
balance	2500

- อธิบายผล

1. ใช้คำสั่ง GET/accounts/{id} ผ่าน Postman เพื่อตรวจสอบข้อมูล Account
2. ระบบค้นหา Account จาก id ที่ระบุ
3. ผลลัพธ์แสดงข้อมูลจาก Account ได้แก่ id, accountNumber, ownerName และ balance
4. ค่า balance ที่แสดงหลังจากการฝากเงินมีจำนวนเพิ่มขึ้นตามจำนวนเงินที่ฝาก
5. ผลการทดลองยืนยันว่าการฝากเงินสามารถเปลี่ยนแปลงยอดคงเหลือของ Account ได้ถูกต้อง

4. ตรวจสอบ DepositTransaction



The screenshot shows the pgAdmin 4 Query Tool interface. The 'Query' tab is active, displaying the SQL query: `SELECT * FROM deposit_tr`. The 'Data Output' tab is also visible, showing the results of the query. The results are displayed in a table with 4 columns: `id` (PK, bigint), `amount` (double precision), and `account_id` (bigint). The first row of data shows `id` as 1, `amount` as 2500, and `account_id` as 1.

	id [PK] bigint	amount double precision	account_id bigint
1	1	2500	1

- อธิบายผล

- 1.เปิด Query Tool ใน pgAdmin4 และเลือก Database lab9
- 2.ใช้คำสั่ง SQL `SELECT * FROM deposit_transaction;` ดังภาพ
- 3.ระบบแสดงข้อมูลรายการฝากเงินที่ถูกบันทึกไว้ในตาราง `deposit_transaction`
- 4.ข้อมูลที่สำคัญ ได้แก่ `id`, `amount`, และ `account_id`
- 5.ค่า `amount` แสดงจำนวนเงินที่ฝาก และ `account_id` แสดง Account ที่เกี่ยวข้องกับรายการฝากเงิน
- 6.ผลการทดลองยืนยันว่าระบบสามารถสร้างและบันทึก DepositTransaction ลงในฐานข้อมูลได้สำเร็จ

5. ทดลอง Rollback

The screenshot displays a REST client interface with two requests and a SQL query editor below.

Request 1: POST /accounts/1/deposit

Method: POST, URL: http://localhost:8080/accounts/1/deposit

Body (JSON):

```
{  "amount": 2500}
```

Response: 500 Internal Server Error (916 ms, 268 B)

Body (JSON):

```
{  "timestamp": "2026-09-09T09:26:44.963Z",  "status": 500,  "error": "Internal Server Error",  "path": "/accounts/1/deposit"}
```

Request 2: GET /accounts/byId

Method: GET, URL: http://localhost:8080/accounts/1

Response: 200 OK (193 ms, 276 B)

Body (JSON):

```
{  "id": 1,  "accountNumber": 6733802894,  "ownerName": "รสริน เมืองหงษ์",  "balance": 2500}
```

SQL Query Editor

Query: `SELECT * FROM deposit_tr`

Data Output:

	id [PK] bigint	amount double precision	account_id bigint
1	1	2500	1

- อธิบายผล

- 1.เพิ่มคำสั่ง `throw new RuntimeException("Test Rollback");` ไว้ท้าย method `deposit()`
- 2.method `deposit()` ยังคงมี `@Transactional` ทำให้การทำงานทั้งหมดอยู่ภายใต้ Transaction เดียวกัน
- 3.เมื่อระบบทำการอัปเดตค่า balance และบันทึก DepositTransaction แล้วเกิด RuntimeException ระบบจะเกิดข้อผิดพลาด
- 4.เนื่องจากมี `@Transactional` ระบบจึงทำการ Rollback การเปลี่ยนแปลงทั้งหมดที่เกิดขึ้นภายใน Transaction
- 5.เมื่อกลับไปตรวจสอบ Account จะพบว่ายอดเงินจะถูก Rollback กลับไปเป็นค่าก่อนเริ่ม Transaction
- 6.เมื่อกลับไปตรวจสอบ deposit_transaction จะไม่พบรายการฝากเงินที่เกิดจากการทดลอง Rollback
- 7.ผลการทดลองแสดงให้เห็นว่า `@Transactional` ช่วยป้องกันไม่ให้ข้อมูลถูกบันทึกเพียงบางส่วนเมื่อเกิดข้อผิดพลาด

6. เปรียบเทียบมี @Transactional กับไม่มี @Transactional

มี @Transactional

Lab9 > accounts_byId

GET http://localhost:8080/accounts/1

Send

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

200 OK • 193 ms • 276 B

{ } JSON Preview Visualize

id	1
accountNumber	6733802894
ownerName	รสริน เมืองหงษ์
balance	2500

Query

Query History

Scratch Pad

1 SELECT * FROM deposit_tr

2

Data Output

Showing rows: 1 to 1

Page No: 1 of 1

	id [PK] bigint	amount double precision	account_id bigint
1	1	2500	1

- อธิบายผล

- 1.method deposit() มี @Transactional กำกับอยู่
- 2.การอัปเดตยอดเงินของ Account และการสร้าง DepositTransaction จะถูกจัดให้อยู่ใน Transaction เดียวกัน
- 3.เมื่อเกิด RuntimeException ระบบจะ Rollback
- 4.ข้อมูลที่ถูกเปลี่ยนแปลงภายใน Transaction จะถูกยกเลิก
- 5.ยอดเงินของ Account จะกลับไปเป็นค่าเดิมและรายการ DepositTransaction ที่เกิดขึ้นในการทดลอง จะไม่ถูกบันทึกค้างไว้
- 6.แสดงให้เห็นว่าการใช้ @Transactional ช่วยรักษาความถูกต้องและความสอดคล้องของข้อมูล

ไม่มี @Transactional

Lab9 > accounts_byId

GET http://localhost:8080/accounts/1

Send

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

200 OK • 175 ms • 276 B

{ } JSON Preview Visualize

id	1
accountNumber	6733802894
ownerName	รสริน เมืองหงษ์
balance	5000

Query

Query History

Scratch Pad

1 SELECT * FROM deposit_tr

2

Data Output

Showing rows: 1 to 2

Page No: 1 of 1

	id [PK] bigint	amount double precision	account_id bigint
1	1	2500	1
2	3	2500	1

- อธิบายผล

1. นำ @Transactional ออกจาก method deposit()
2. ยังคงคำสั่ง throw new RuntimeException("Test Rollback"); เพื่อจำลองข้อผิดพลาด
3. เมื่อเกิด Exception จะไม่มี Transaction เดียวที่ครอบคลุมการทำงานทั้งหมดของ method
4. การเปลี่ยนแปลงที่เกิดขึ้นก่อนเกิด Exception ถูกบันทึกลงในฐานข้อมูลไปแล้ว
5. ข้อมูลบางส่วนอาจยังคงอยู่ แม้ว่าการทำงานโดยรวมจะเกิดข้อผิดพลาด
6. ทำให้เกิดข้อมูลไม่สอดคล้องกันได้ เช่น ยอดเงินถูกเปลี่ยนแปลง แต่รายการธุรกรรมไม่ถูกบันทึก หรือเกิดข้อผิดพลาดจากการทำงานบางส่วน

7. คำถามท้าย Lab

7.1. @Transactional มีหน้าที่อะไร?

-> กำหนดให้การทำงานของเมธอดอยู่ภายใต้ Transaction เดียวกัน โดยการเปลี่ยนแปลงข้อมูลจะถูกดำเนินการร่วมกัน หากการทำงานสำเร็จทั้งหมดจึงจะ COMMIT ข้อมูลลง Database แต่ถ้าเกิด Error หรือ Exception ระบบสามารถ ROLLBACK การเปลี่ยนแปลงที่เกิดขึ้นภายใน Transaction ได้

7.2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

-> เนื่องจากการเปลี่ยนแปลงข้อมูลมากกว่าหนึ่งส่วน ได้แก่ การเพิ่มยอด(balance) และการบันทึกข้อมูล (DepositTransaction) ถ้าเกิด error ระหว่างการทำงาน การใช้ Transaction จะช่วยให้สามารถยกเลิกการเปลี่ยนแปลงทั้งหมดได้ ทำให้ข้อมูลใน database ยังคงถูกต้องและสอดคล้องกัน

7.3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

-> commit คือ การยืนยันการเปลี่ยนแปลงข้อมูลใน Transaction และบันทึกข้อมูลลงใน database อย่างถาวร ส่วน rollback คือ การยกเลิกการเปลี่ยนแปลงข้อมูลใน Transaction และทำให้ข้อมูลกลับไปเป็นสถานะก่อนเริ่ม Transaction ดังนั้น commit ใช้เมื่อการทำงานเสร็จ ส่วน rollback ใช้เมื่อเกิดข้อผิดพลาดและต้องการยกเลิกการเปลี่ยนแปลง

7.4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

-> Many-to-One โดย Account หนึ่งสามารถมีรายการเงินฝากได้หลายรายการ แต่แต่ละรายการจะเชื่อมโยงกับ Account เพียงหนึ่งบัญชี

7.5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

-> ระบบจะทำการ rollback การเปลี่ยนแปลงที่เกิดขึ้นภายใน Transaction ทั้งหมด ทำให้ database กลับไปเป็นสถานะก่อนเริ่ม

7.6. จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

-> ไม่มี Transaction ที่ครอบคลุมการอัปเดต Account และการสร้าง DepositTransaction เมื่อเกิด Error การเปลี่ยนแปลงที่เกิดขึ้นก่อนหน้าอาจถูกบันทึกลงใน Database ไปแล้ว และไม่สามารถ Rollback การทำงานทั้งหมดกลับพร้อมกันได้ สิ่งที่จะเกิดข้อมูลไม่สอดคล้องกัน เช่น ค่า balance ถูกเปลี่ยนแปลง แต่ DepositTransaction ไม่ถูกบันทึก

สรุปผลการทดลอง

จากการทดลอง พบว่า Transaction มีความสำคัญต่อการทำงานที่ต้องเปลี่ยนแปลงข้อมูลหลายส่วนในฐานข้อมูล โดยเฉพาะกรณีการฝากเงิน ซึ่งต้องทำทั้งการเพิ่มยอด balance และการสร้าง

DepositTransaction การใช้ @Transactional ทำให้การทำงานทั้งสองส่วนอยู่ใน Transaction เดียวกัน

หากเกิดข้อผิดพลาดขึ้น ระบบก็สามารถ Rollback การเปลี่ยนแปลงกลับไปยังสถานะก่อนเริ่ม

Transaction ได้ ช่วยป้องกันการเกิดข้อมูลที่ไม่สมบูรณ์หรือไม่สอดคล้องกันในฐานข้อมูล